

Ultimate Java Cheatsheet For Exams

From Basics to Advanced

1. Java Basics

Java Program Structure

```
class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello Java");  
    }  
}
```

Memory Hack

- `class` = blueprint
- `main()` = entry point
- `System.out.println()` = print line

Golden Rule

Java program always starts from:

```
public static void main(String[] args)
```

2. Variables

Syntax

```
int age = 21;  
float marks = 88.5f;  
double salary = 45000.50;  
char grade = 'A';  
boolean isPass = true;  
String name = "Avani";
```

Primitive Data Types

Type	Size	Example
byte	1 byte	<code>byte a = 10;</code>
short	2 byte	<code>short s = 100;</code>
int	4 byte	<code>int x = 50;</code>
long	8 byte	<code>long n = 9999L;</code>
float	4 byte	<code>float f = 2.5f;</code>
double	8 byte	<code>double d = 5.55;</code>
char	2 byte	<code>char c = 'A';</code>
boolean	true/false	<code>boolean b = true;</code>

Easy Trick

- float -> add `f`
- long -> add `L`
- char -> single quotes
- String -> double quotes

3. Input In Java

Scanner Class

```
import java.util.Scanner;

Scanner sc = new Scanner(System.in);

int age = sc.nextInt();
float marks = sc.nextFloat();
String name = sc.next();
String full = sc.nextLine();
```

Memory Hack

- `next()` -> single word
 - `nextLine()` -> full sentence
-

4. Operators

Arithmetic Operators

+ - * / %

Relational Operators

> < >= <= == !=

Logical Operators

&& || !

Increment/Decrement

```
x++
++x
x--
--x
```

Most Asked Viva

```
int x = 5;
System.out.println(x++); // 5
System.out.println(++x); // 7
```

Trick

- Post -> use then increase
 - Pre -> increase then use
-

5. Conditional Statements

if Statement

```
if(age >= 18) {
    System.out.println("Adult");
}
```

if else

```
if(marks >= 40) {
    System.out.println("Pass");
} else {
    System.out.println("Fail");
}
```

else if Ladder

```
if(marks >= 90) {
    System.out.println("A");
}
```

```
else if(marks >= 75) {  
    System.out.println("B");  
}  
else {  
    System.out.println("C");  
}
```

switch

```
switch(day) {  
    case 1:  
        System.out.println("Monday");  
        break;  
  
    case 2:  
        System.out.println("Tuesday");  
        break;  
  
    default:  
        System.out.println("Invalid");  
}
```

Memory Hack

- **break** stops execution
 - **default** works like else
-

6. Loops

for Loop

```
for(int i=1; i<=5; i++) {  
    System.out.println(i);  
}
```

while Loop

```
int i = 1;
while(i <= 5) {
    System.out.println(i);
    i++;
}
```

do while

```
int i = 1;

do {
    System.out.println(i);
    i++;
} while(i <= 5);
```

Loop Difference

Loop	Condition Check
for	before
while	before
do while	after

Important Keywords

break;
continue;

Trick

- break -> exit loop
- continue -> skip current iteration

7. Arrays

Array Declaration

```
int arr[] = {10,20,30,40};
```

Access Array

```
System.out.println(arr[0]);
```

Loop Through Array

```
for(int i=0; i<arr.length; i++) {  
    System.out.println(arr[i]);  
}
```

Enhanced for Loop

```
for(int x : arr) {  
    System.out.println(x);  
}
```

Memory Hack

- Array index starts from 0
 - `length` has no brackets
-

8. Methods

Method Syntax

```
static void greet() {  
    System.out.println("Hello");  
}
```

Method With Return

```
static int add(int a, int b) {  
    return a + b;  
}
```

Method Calling

```
greet();  
int ans = add(10,20);
```

Memory Formula

```
returnType methodName(parameters)
```

9. Constructors

Default Constructor

```
class Student {  
    Student() {  
        System.out.println("Constructor Called");  
    }  
}
```

Parameterized Constructor

```
class Student {  
    int id;  
  
    Student(int x) {  
        id = x;  
    }  
}
```

Memory Hack

Constructor name = class name

10. this Keyword

Syntax

```
class Student {  
    int id;  
  
    Student(int id) {  
        this.id = id;  
    }  
}
```

Trick

- **this** refers current object
 - Used to remove variable confusion
-

11. static Keyword

Static Variable

```
class Test {  
    static int count = 0;  
}
```

Static Method

```
static void show() {  
    System.out.println("Static Method");  
}
```

Important Rule

Static members belong to class, not object.

12. final Keyword

final Variable

```
final int x = 10;
```

final Method

Cannot override.

final Class

Cannot inherit.

Memory Trick

final = fixed forever

13. Inheritance

Syntax

```
class Parent {  
    void show() {  
        System.out.println("Parent");  
    }  
}
```

```
class Child extends Parent {  
}
```

Types

- Single
- Multilevel
- Hierarchical

Not Supported

Multiple inheritance using classes.

14. Method Overloading

Same Method Name Different Parameters

```
class Test {  
    void add(int a, int b) {  
    }  
  
    void add(int a, int b, int c) {  
    }  
}
```

Trick

Compile time polymorphism.

15. Method Overriding

Same Method Same Parameters

```
class Parent {  
    void show() {  
        System.out.println("Parent");  
    }  
}
```

```
}  
  
class Child extends Parent {  
    void show() {  
        System.out.println("Child");  
    }  
}
```

Trick

Runtime polymorphism.

16. super Keyword

Call Parent Constructor

```
super();
```

Access Parent Variable

```
super.x;
```

Access Parent Method

```
super.show();
```

17. Abstraction

Abstract Class

```
abstract class Animal {  
    abstract void sound();  
}
```

Important Rule

Cannot create object of abstract class.

18. Interface

Syntax

```
interface Demo {  
    void show();  
}  
  
class Test implements Demo {  
    public void show() {  
        System.out.println("Hello");  
    }  
}
```

Memory Hack

- interface = blueprint only
- implements keyword used

Important Points

- Methods are public and abstract by default
 - Variables are public static final
-

19. Packages

Create Package

```
package mypack;
```

Import Package

```
import java.util.Scanner;
```

Memory Trick

Package = folder of classes

20. Exception Handling

try catch

```
try {  
    int x = 10/0;  
}  
catch(Exception e) {  
    System.out.println(e);  
}
```

finally

```
finally {  
    System.out.println("Always Executes");  
}
```

throw

```
throw new ArithmeticException();
```

throws

```
void test() throws IOException {  
}
```

Memory Hack

- throw -> manually create exception
 - throws -> declare exception
-

21. Multithreading

Create Thread Using Thread Class

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println("Thread Running");  
    }  
}
```

Start Thread

```
MyThread t = new MyThread();  
t.start();
```

Using Runnable

```
class Demo implements Runnable {  
    public void run() {  
        System.out.println("Run Method");  
    }  
}
```

22. Thread Methods

sleep()

```
Thread.sleep(1000);
```

join()

```
t.join();
```

setName()

```
t.setName("MyThread");
```

getName()

```
System.out.println(t.getName());
```

23. Synchronization

Synchronized Method

```
synchronized void show() {  
}
```

Synchronized Block

```
synchronized(this) {  
}
```

Memory Hack

Synchronization avoids data inconsistency.

24. String

Create String

```
String s = "Java";
```

Important Methods

```
s.length();  
s.charAt(0);  
s.toUpperCase();  
s.toLowerCase();  
s.substring(1,3);  
s.equals("Java");
```

String Comparison

```
s.equals(t);
```

Important Viva

Never use `==` for content comparison.

25. StringBuffer

Syntax

```
StringBuffer sb = new StringBuffer("Java");
```

Methods

```
sb.append(" Code");  
sb.insert(1,"X");  
sb.delete(1,3);  
sb.reverse();
```

Trick

Mutable and thread safe.

26. StringBuilder

Syntax

```
StringBuilder sb = new StringBuilder("Java");
```

Difference

StringBuffer	StringBuilder
--------------	---------------

Thread safe	Not thread safe
-------------	-----------------

Slow	Fast
------	------

27. Wrapper Classes

Autoboxing

```
Integer a = 10;
```

Unboxing

```
int x = a;
```

Important Wrapper Classes

- Integer
- Float
- Double
- Character
- Boolean

28. Collections Framework

ArrayList

Syntax

```
ArrayList<Integer> list = new ArrayList<>();
```

Methods

```
list.add(10);  
list.remove(0);  
list.get(0);  
list.size();
```

LinkedList

```
LinkedList<String> list = new LinkedList<>();
```

HashSet

```
HashSet<Integer> set = new HashSet<>();
```

Important

Does not allow duplicates.

HashMap

```
HashMap<Integer,String> map = new HashMap<>();
```

Methods

```
map.put(1,"Java");  
map.get(1);  
map.remove(1);
```

29. File Handling

Create File

```
File f = new File("test.txt");
```

FileWriter

```
FileWriter fw = new FileWriter("test.txt");  
fw.write("Hello");  
fw.close();
```

FileReader

```
FileReader fr = new FileReader("test.txt");
```

30. JDBC Basics

Steps

1. Import package
2. Load driver
3. Create connection
4. Create statement
5. Execute query
6. Close connection

Connection Syntax

Connection con = DriverManager.getConnection(url,user,password);

31. Important OOP Concepts

Concept	Meaning
Encapsulation	Wrapping data
Inheritance	Acquiring properties
Polymorphism	Many forms
Abstraction	Hiding implementation

32. Access Modifiers

Modifier	Same Class	Same Package	Subclasses	Other Package
----------	------------	--------------	------------	---------------

private	Yes	No	No	No
default	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

33. Important Keywords

Keyword	Use
static	class level
final	fixed
this	current object
super	parent class
extends	inheritance
implements	interface
synchronized	thread safety
abstract	incomplete class

34. Frequently Asked Output Questions

Q1

```
String s = "Java";  
System.out.println(s.charAt(2));
```

Output

v

Q2

```
int x = 5;  
System.out.println(x++ + ++x);
```

Output

12

Q3

```
String s1 = "Java";  
String s2 = "Java";  
  
System.out.println(s1 == s2);
```

Output

true

35. Most Important Viva Questions

Difference Between == and equals()

- == checks reference
- equals() checks content

Difference Between StringBuffer and StringBuilder

- StringBuffer is synchronized
- StringBuilder is faster

Difference Between Method Overloading and Overriding

- Overloading -> compile time
- Overriding -> runtime

Why Java Is Platform Independent?

Because of JVM.

Why Multiple Inheritance Not Supported?

To avoid ambiguity problem.

36. Exam Quick Revision Tricks

OOP Memory Formula

EIPA

- Encapsulation
 - Inheritance
 - Polymorphism
 - Abstraction
-

Loop Formula

Initialization → Condition → Update

Array Formula

Datatype[] name = {values}

Method Formula

returnType methodName(parameters)

Class Formula

```
class ClassName {  
}
```

37. One Day Before Exam Strategy

Must Practice Topics

- Arrays
- String programs
- OOP concepts
- Constructors
- Inheritance
- Exception handling
- Multithreading basics
- Collections

Most Repeated Programs

- Palindrome
 - Prime number
 - Factorial
 - Fibonacci
 - Sorting array
 - String reverse
 - Method overloading
 - Constructor program
 - Exception handling
 - Thread creation
-

38. Golden Rules For Exams

Theory

- Write definitions in simple words
- Draw diagrams for OOP
- Mention advantages whenever possible

Programs

- First write algorithm
- Then syntax
- Add comments if possible
- Don't forget semicolon

Viva

- Speak confidently
 - Explain line by line
 - Remember why we use that concept
-

39. Super Short Revision Notes

String

Immutable

StringBuffer

Mutable + thread safe

StringBuilder

Mutable + fast

Array

Fixed size

ArrayList

Dynamic size

Interface

100% abstraction

Abstract Class

Partial abstraction

JVM

Runs bytecode

JDK

Development kit

JRE

Runtime environment

40. Final 5 Minute Revision

Most Important Syntaxes

Scanner

```
Scanner sc = new Scanner(System.in);
```

Array

```
int arr[] = {1,2,3};
```

Loop

```
for(int i=0; i<n; i++)
```

Method

```
static void show()
```

Constructor

```
ClassName()
```

Inheritance

```
class B extends A
```

Interface

```
class Test implements Demo
```

Exception

```
try {} catch(Exception e) {}
```

Thread

```
class MyThread extends Thread
```

ArrayList

```
ArrayList<Integer> list = new ArrayList<>();
```

Final Motivation

You do not need to remember every line.

Understand the pattern and syntax flow.

Most Java programs follow the same structure again and again.

If you know:

- class
- main method
- loops
- methods
- OOP concepts
- arrays
- strings

then honestly you can solve most university exam questions pretty easily.

All the best for exams 🚀

41. Recursion

Basic Syntax

```
static void print(int n) {  
    if(n == 0)  
        return;  
  
    System.out.println(n);  
    print(n-1);  
}
```

Memory Hack

Function calling itself = recursion

Golden Rule

Every recursion must have:

- base condition
 - recursive call
-

42. Important Recursion Programs

Factorial

```
static int fact(int n) {  
    if(n == 1)  
        return 1;  
  
    return n * fact(n-1);  
}
```

Fibonacci

```
static int fib(int n) {  
    if(n <= 1)  
        return n;  
  
    return fib(n-1) + fib(n-2);  
}
```

43. Command Line Arguments

Syntax

```
class Test {  
    public static void main(String[] args) {  
        System.out.println(args[0]);  
    }  
}
```

Run

```
java Test Hello
```

Output

```
Hello
```

44. Type Casting

Widening Casting

```
int x = 10;  
double y = x;
```

Narrowing Casting

```
double x = 10.5;  
int y = (int)x;
```

Trick

Small to big = automatic
Big to small = manual

45. Arrays Programs Shortcuts

Find Sum

```
int sum = 0;  
  
for(int x : arr) {  
    sum += x;  
}
```

Find Maximum

```
int max = arr[0];  
  
for(int x : arr) {  
    if(x > max)  
        max = x;  
}
```

Reverse Array

```
for(int i=arr.length-1; i>=0; i--) {  
    System.out.println(arr[i]);  
}
```

46. Pattern Programs

Star Pattern

```
for(int i=1; i<=5; i++) {  
    for(int j=1; j<=i; j++) {  
        System.out.print("*");  
    }  
  
    System.out.println();  
}
```

Output

```
*  
**  
***  
****  
*****
```

47. Important Number Programs

Palindrome Number

```
int n = 121;  
int temp = n;  
int rev = 0;  
  
while(n > 0) {  
    int r = n % 10;  
    rev = rev * 10 + r;  
    n = n / 10;  
}  
  
if(temp == rev)  
    System.out.println("Palindrome");
```

Prime Number

```
int n = 7;
boolean prime = true;

for(int i=2; i<n; i++) {
    if(n % i == 0) {
        prime = false;
        break;
    }
}

if(prime)
    System.out.println("Prime");
```

48. String Programs

Reverse String

```
String s = "Java";
String rev = "";

for(int i=s.length()-1; i>=0; i--) {
    rev += s.charAt(i);
}

System.out.println(rev);
```

Palindrome String

```
String s = "madam";
String rev = "";

for(int i=s.length()-1; i>=0; i--) {
    rev += s.charAt(i);
}

if(s.equals(rev))
    System.out.println("Palindrome");
```

49. 2D Arrays

Syntax

```
int arr[][] = {  
    {1,2,3},  
    {4,5,6}  
};
```

Traversing

```
for(int i=0; i<arr.length; i++) {  
    for(int j=0; j<arr[i].length; j++) {  
        System.out.print(arr[i][j] + " ");  
    }  
  
    System.out.println();  
}
```

Memory Hack

```
Rows = arr.length  
Columns = arr[i].length
```

50. Important String Methods

Method	Use
length()	total characters
charAt()	get character
substring()	get part of string

<code>equals()</code>	compare content
<code>contains()</code>	check word
<code>replace()</code>	replace text
<code>trim()</code>	remove spaces
<code>split()</code>	divide string

51. Important Collection Differences

Array	ArrayList
Fixed size	Dynamic size
Fast	Flexible
Primitive allowed	Objects only

HashSet	ArrayList
No duplicates	Duplicates allowed
Unordered	Ordered

HashMap	HashSet
Key value pair	Only values
Uses <code>put()</code>	Uses <code>add()</code>

52. Java Memory Areas

Area	Purpose
Heap	Objects

Stack	Methods and local variables
Method Area	Class information

53. Garbage Collection

Syntax

`System.gc();`

Meaning

Unused objects are removed from memory.

54. Important Java Terminologies

Term	Meaning
JVM	Java Virtual Machine
JDK	Java Development Kit
JRE	Java Runtime Environment
Bytecode	.class file
Compiler	converts code to bytecode

55. Java Keywords Quick Revision

Keyword	Meaning
new	create object
class	blueprint
object	instance of class
return	send value back
void	no return
public	accessible everywhere
private	accessible only inside class
protected	accessible in inheritance
package	group of classes
import	use package

56. Constructor Vs Method

Constructor	Method
Same name as class	Any name
No return type	Has return type
Called automatically	Called manually
Used for initialization	Used for operations

57. Compile Time Vs Runtime

Compile Time	Runtime
Before execution	During execution
Syntax errors	Logical errors
Method overloading	Method overriding

58. Checked Vs Unchecked Exception

Checked	Unchecked
Checked by compiler	Not checked by compiler
IOException	ArithmeticException
Must handle	Optional handling

59. Java 8 Important Concepts

Lambda Expression

Syntax

(a,b) -> a+b

Example

```
interface Demo {  
    void show();  
}
```

```
Demo d = () -> System.out.println("Hello");  
d.show();
```

Functional Interface

Syntax

```
@FunctionalInterface
interface Demo {
    void show();
}
```

Rule

Only one abstract method.

60. Stream API Basics

Example

```
ArrayList<Integer> list = new ArrayList<>();
```

```
list.stream()
    .filter(x -> x % 2 == 0)
    .forEach(System.out::println);
```

Common Methods

- filter()
 - map()
 - sorted()
 - count()
 - forEach()
-

61. Important Interview Programs

Swap Without Third Variable

```
int a = 10;  
int b = 20;
```

```
a = a + b;  
b = a - b;  
a = a - b;
```

Find Largest Of 3 Numbers

```
int a = 10, b = 20, c = 30;
```

```
if(a>b && a>c)  
    System.out.println(a);  
else if(b>c)  
    System.out.println(b);  
else  
    System.out.println(c);
```

62. Common Errors Students Make

Missing Semicolon

```
System.out.println("Hello")
```

Wrong Main Method

```
public static void main(String args)
```

Using == For Strings

```
s1 == s2
```

Forgetting break In switch

Array Index Out Of Bounds

arr[10]

63. Quick Debugging Tricks

If Output Not Coming

- Check loop condition
- Check method calling
- Check semicolon
- Check variable values

If Compilation Error

- Missing bracket
 - Missing semicolon
 - Wrong spelling
 - Wrong datatype
-

64. Java Program Writing Format For Exams

Step 1

Write algorithm

Step 2

Write class name

Step 3

Write main method

Step 4

Write logic step by step

Step 5

Show output properly

65. Ultimate Last Minute Revision Table

Topic	Most Important Thing
Loops	condition and update
Arrays	index starts from 0
OOP	understand real life example
Constructor	automatic call
Inheritance	extends keyword
Interface	implements keyword
Exception	try catch
Thread	start() method
Collections	ArrayList and HashMap
String	immutable

66. 20 Super Important One Liners

- Java is platform independent.
 - JVM executes bytecode.
 - Object is instance of class.
 - Constructor has no return type.
 - Java does not support multiple inheritance using classes.
 - String objects are immutable.
 - Array index starts from 0.
 - Method overloading is compile time polymorphism.
 - Method overriding is runtime polymorphism.
 - Interface supports abstraction.
 - Abstract class cannot create objects.
 - finally block always executes.
 - Collection framework stores dynamic data.
 - HashSet does not allow duplicates.
 - HashMap stores key value pairs.
 - Threads enable multitasking.
 - Synchronization prevents inconsistency.
 - Garbage collector removes unused objects.
 - Scanner is used for input.
 - Java is object oriented language.
-